File Management

Lecture 1

The Big Picture

The Problem:
The computer's main memory (RAM) is volatile (its contents are lost when power is off) and too small to store all data permanently. We need a system to store, organize, and manipulate data on non-volatile storage devices like hard drives, SSDs, and USB drives.
The Operating System provides a logical, user-friendly abstraction for physical storage devices. This abstraction is the file system. Its primary goals are:

- User Convenience: To hide the complexities of disk geometry and low-level I/O.
- Efficiency: To manage disk space effectively, minimizing fragmentation and access time.
- Logical Organization: To provide a structure (directories) for users to organize their data logically.

File Concepts

The fundamental logical unit of storage in an OS is the file.

- Definition: A file is a named collection of related information stored on secondary storage.
- Abstraction: The OS maps files onto physical devices. A file is a sequence of logical bytes, regardless of where they physically reside on the disk.

File Naming:

- Syntax: Usually consists of a name and an extension (e.g., `lecture.pdf`).
- Role of Extension: used to indicate the file type and the program that can open it (e.g., `.txt`, `.exe`, `.c`, `.jpg`). In some systems, the extension is just a convention; in others (like Windows), it's crucial for the OS to determine the file type.

Extension	Meaning
.bak	Backup file
.c	C source program
.gif	Compuserve Graphical Interchange Format image
.html	World Wide Web HyperText Markup Language document
.jpg	Still picture encoded with the JPEG standard
.mp3	Music encoded in MPEG layer 3 audio format
.mpg	Movie encoded with the MPEG standard
.o	Object file (compiler output, not yet linked)
.pdf	Portable Document Format file
.ps	PostScript file
.tex	Input for the TEX formatting program
.txt	General text file
.zip	Compressed archive

Figure 4-1. Some typical file extensions.

File Concepts

File Structure

Files can be structured in three ways:

1. Byte Stream (Unstructured): The simplest and most common form. The OS views the file as a simple sequence of bytes. It imposes no structure; any meaning (e.g., lines, records) is imposed by the application. Used by UNIX, Linux, and Windows.
2. Record Sequence: The file is a collection of fixed-length or variable-length records. Used primarily in older mainframe systems (like IBM's CMS).
3. Tree of Records: A more complex structure where records are organized in a tree hierarchy, used in some mainframe database systems.

File attributes and Operations

The OS stores information about each file in a data structure called the File Control Block (FCB) or inode.

Common File Attributes:

- Name: The symbolic name known to the user.
- Identifier (ID): A unique, system-wide tag (often a number) that identifies the file within the file system.
- Type: Information needed for systems that support different file types.
- Location: A pointer to the location of the file on the device (e.g., block numbers on a disk).
- Size: The current size of the file (and possibly the maximum allowed size).
- Protection: Access control information (who can read, write, execute).
- Timestamps: Creation time, last modification time, last access time.
- Ownership: Information about the user who owns the file.

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file was last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Figure 4-5. Some possible file attributes.

File attributes and Operations

Basic File Operations:

The OS provides system calls for these fundamental operations:

1. Create: Allocate space and create an entry in the directory.
2. Open: A preparatory step that copies the file's attributes (FCB) into main memory for faster access, returning a file descriptor or handle.
3. Read: Read data from the file. Requires a pointer to indicate where to start.
4. Write: Write data to the file, usually at the current file pointer location.
5. Seek: Reposition the file pointer within the file.
6. Delete: Release the file's space and remove the directory entry.
7. Close: Destroy the in-memory copy of the FCB (the handle) and complete all pending writes.

File Access

How does a program access the data inside a file?

1. Sequential Access: The simplest method. Data is processed in order, one record after another.
    - *Analogy:* A cassette tape.
    - *Operations:* `read next`, `write next`. The file pointer is automatically advanced.
    - *Use case:* Editors, compilers.
2. Direct Access (or Random Access): Files are made up of fixed-length logical records. The program can read records in any order.
    - *Analogy:* A CD or record player.
    - *Operations:* `read n`, `write n`, where `n` is the logical record number.
    - *Use case:* Databases, real-time systems.
3. Indexed Access: Built on top of direct access. An index contains pointers to various blocks. To find a record, you first search the index, then access the file directly.
    - *Use case:* Very large databases.

Directory Structure

A directory is a symbol table that maps file names to their corresponding FCBs.

The structure of the directory determines how users can organize their files.

a) Single-Level Directory

- Structure: One directory containing all files from all users.
- Pros: Simple, easy to understand.
- Cons: Naming collision (two users can't have the same file name), very poor organization.

b) Two-Level Directory

- Structure: A separate directory for each user, plus a Master File Directory (MFD) that lists user directories.
- Pros: Solves naming collisions (User A's `temp.txt` is different from User B's `temp.txt`). Basic isolation.
- Cons: Users can't group their *own* files into sub-categories. Isolation can make sharing difficult.

Directory Structure

c) Tree-Structured Directories

- Structure: Users can create subdirectories, leading to a tree (or hierarchy). This is the most common structure in use today (Linux, macOS, Windows).
- Concepts:
  - Current Directory (Working Directory): To avoid typing the full pathname every time, users can designate a current directory. Paths can be absolute (from the root, e.g., `/home/user/proj/main.c`) or relative (from the current directory, e.g., `proj/main.c`).
- Pros: Natural, powerful, allows deep organization.

Directory Operations

Operation	Description
Create	Make a new empty directory (e.g., mkdir)
Delete	Remove an empty directory (e.g., rmdir)
OpenDir	Open a directory for reading (returns a handle)
ReadDir	Read next entry (filename + metadata)
CloseDir	Release directory handle
Link	Create a hard or symbolic link to a file
Unlink	Remove a directory entry (decrements link count; file deleted if count = 0)

Sharing and Protection

Sharing

When multiple users or processes share files, the OS must manage concurrency and consistency.

- Simultaneous Access: If multiple users try to write to the same file at the same time, the results can be disastrous.
- Solution: Semaphores/Mutexes: The OS may provide mechanisms for file locking (mandatory or advisory) to prevent race conditions.

Sharing and Protection

Protection

The file system must implement a protection scheme to control access.

1. Password-based: The file is protected by a password. Simple but inconvenient.
2. Access Control Lists (ACL): A detailed list associated with each file, specifying exactly which users/groups can perform which operations. Highly flexible but can be complex to manage.
3. UNIX/Linux "Mode" Bits: A compact 3-tiered approach:
    - User: The owner of the file.
    - Group: A group of users who share the same permissions.
    - Other (World): Everyone else.
    - For each tier, permissions are defined as Read (r), Write (w), Execute (x).

References

Chapter 4 of A. S. Tanenbaum, Modern Operating Systems, 5th ed. Pearson Education, Inc., 2022